TLS Server-Identity Validation Disabled by Default in `lightway-client`

Security Finding & Proof of Concept

ExpressVPN Bug Bounty Program — YesWeHack

Scope: github.com/expressvpn/lightway, github.com/expressvpn/wolfssl-rs

Author: **L&_M@z4rt**

Date: 2026-10-22

CWE-295 / CWE-297 — Improper Certificate / Hostname Validation

Program: ExpressVPN Bug Bounty (YesWeHack) **Scope assets:** `github.com/expressvpn/lightway`, `github.com/expressvpn/wolfssl-rs` **Author:** L&_M@z4rt **Date:** 2026-10-22 **Vulnerability class:** CWE-295 (Improper Certificate Validation) / CWE-297 (Improper Validation of Certificate with Host Mismatch) **Status:** Confirmed in the open-source reference implementation via three reproducible regression tests. Not yet confirmed against the compiled, distributed ExpressVPN application — see *Open Question for the Vendor*.

---

1. Executive Summary

`lightway-client`, and the `lightway-core` / `wolfssl-rs` libraries it is built on, correctly validate that a VPN server's TLS certificate chains to a trusted CA. They **never validate that the certificate's identity (CN / SAN) actually matches the server being connected to**, unless the operator explicitly supplies a `server_dn` value (CLI flag `--server-dn`, config field `server_dn`). This field is **empty by default**, and nothing in the codebase warns the operator when it is left unset.

Practical effect: anyone holding a certificate signed by the CA the client trusts — for example the private key of one compromised node in a server fleet, or any other certificate validly issued under that same CA — can present it while claiming to be **any other server** trusted under that CA, and the client will accept it as genuine. Chain-of-trust validation passes; the "is this specifically the host I meant to reach" check is simply never performed.

Three regression tests built against the real, vendored wolfSSL library (not a mock) confirm this precisely, and rule out the more severe "no validation at all" hypothesis along the way.

---

2. Root Cause

2.1 The check is opt-in at the `lightway-core` API level

`lightway-core/src/connection/builders.rs:172-178`:

```
 /// Enable server domain name validation
pub fn with_server_domain_name_validation(self, server_dn: &str) -> Self {
    Self {
        session_config: self.session_config.with_checked_domain_name(server_dn),
        ..self
    }
}
```

`wolfssl-rs/wolfssl/src/ssl.rs` declares the underlying field as:

```
 /// If set, configures the session to check the given domain against the
/// peer certificate during connection.
pub checked_domain_name: Option<String>,
```

If `None`, no identity/hostname check is ever installed on the session — the certificate's CN and SAN entries are never compared against anything.

2.2 `lightway-client` only calls it conditionally

`lightway-client/src/lib.rs:1503-1506`:

```
 .when_some(server_dn, |b, sdn| {
    b.with_server_domain_name_validation(&sdn)
})
```

`server_dn: Option<String>` is threaded from the CLI/config layer:

- `lightway-client/src/config.rs:70` — `pub server_dn: String` (CLI flag `--server-dn`)
- `lightway-client/src/config.rs:544` — default value: `server_dn: String::new()` (**empty**)
- `lightway-client/src/config.rs:411-412`: `rust server_dn: (! self.server_dn.is_empty()) .then(|| std::mem::take(&mut self.server_dn)),` An empty string becomes `None`, which skips the check entirely.

`Config::validate()` in the same file checks several other settings for correctness (buffer sizes, device GUID format, TUN queue length, etc.) and emits warnings or hard errors for misconfiguration — **but has no check and emits no warning when `server_dn` is left unset.**

2.3 Verify-mode is never set explicitly either

An exhaustive search across `lightway-core` and `lightway-client` shows **zero** calls to `with_ssl_verify_mode` / `wolfSSL_set_verify`. TLS verify-mode behavior therefore comes entirely from wolfSSL's own default. This was empirically confirmed (Test 1, below) to still enforce CA-chain validation — but chain validation and hostname/identity validation are two independent checks in wolfSSL's API, and only the first happens unconditionally.

---

3. Proof of Concept

Three regression tests were added to `wolfssl-rs/wolfssl/src/ssl.rs` (`mod tests`), compiled and executed against the real vendored wolfSSL C library via `cargo test`, with **no modification to any production code path** — only new test functions.

Test 1 — Sanity check: chain-of-trust validation *is* enforced by default

Rules out the more severe hypothesis that TLS verification is entirely disabled.

Setup: client loads a legitimate CA (`CA_CERT` test fixture); `with_ssl_verify_mode` is never called (matching `lightway-client`'s real code path exactly). The server presents a **self-signed certificate from a completely unrelated, attacker-generated CA**:

```
openssl req -x509 -newkey rsa:2048 -keyout rogue_key.pem -out rogue_cert.pem \
  -days 365 -nodes -subj "/CN=evil.attacker.example"
```

Result:

```
 client_err = Some("Fatal: CA cert not available to verify certificate")
client finished = false
```

→ **Correctly rejected.** No full bypass of TLS verification exists.

Test 2 — The finding: identity/hostname checking is skipped by default

Setup: same client context (legitimate CA loaded, no verify-mode call). The server presents `SERVER_CERT`, a certificate **validly signed by the loaded CA** (`CN=www.wolfssl.com`, `SAN=DNS:example.com, IP:127.0.0.1`) — matching neither the CN nor any SAN entry of the server the client is actually trying to reach. `with_checked_domain_name` is **never called**, mirroring `lightway-client` with its default, empty `server_dn`.

Result:

```
 client finished (identity mismatch, no domain check requested) = true
```

→ **Handshake completes anyway.** The client has no way to detect it is talking to the wrong server.

Test 3 — Control: the feature works correctly when actually enabled

Identical setup to Test 2, except `with_checked_domain_name("attacker-controlled-host.test")` is explicitly set, simulating `--server-dn` being passed by the operator.

Result:

```
 client_err (domain check enabled, mismatched name) = Some("Fatal: Domain name mismatch")
```

→ **Correctly rejected.** Confirms the mechanism itself works; it is simply off by default, silently.

3.1 Reproduction steps

```
 git clone https://github.com/expressvpn/wolfssl-rs
cd wolfssl-rs
git submodule update --init --depth 1

# Generate the rogue, CA-unrelated certificate used in Test 1
openssl req -x509 -newkey rsa:2048 -keyout /tmp/rogue_key.pem -out /tmp/rogue_cert.pem \
  -days 365 -nodes -subj "/CN=evil.attacker.example"
# Convert to DER, then to the crate's include!-able Rust byte-array fixture
# format, dropped at wolfssl/tests/data/rogue_cert_der and rogue_key_der.

# Apply the attached patch (ssl_rs_poc.patch) to wolfssl/src/ssl.rs

cd wolfssl
cargo test --lib default_verify_mode_with_rogue_unrelated_cert -- --nocapture
cargo test --lib default_verify_mode_skips_hostname_check_when_not_requested -- --nocapture
cargo test --lib checked_domain_name_rejects_identity_mismatch_when_enabled -- --nocapture
```

Full diff of the three test functions is provided as `ssl_rs_poc.patch`.

4. Impact

Check	Enforced by default?
Certificate chains to a trusted CA	**Yes**
Certificate identity (CN/SAN) matches the intended server	**No**

For a VPN provider operating many server nodes under one (or a small number of) issuing CA(s), this means: obtaining a single validly-signed certificate under that CA — through compromise of one node's private key, an insider, a CA-issuance-pipeline compromise, or any other misissuance — is sufficient to **impersonate any other server trusted under that CA** to a client, without needing to break TLS cryptography or compromise the client's actual intended destination at all.

This degrades the security guarantee from *"I am talking to this specific server"* to *"I am talking to some server this CA has ever vouched for"* — a meaningful weakening for a product whose core value proposition is that the specific server handling a user's traffic is a trust-sensitive choice.

Estimated severity: Medium–High. **CVSS 3.1 estimate:** `AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:N` ≈ **6.5–7.0** (`AC:H` because exploitation requires the attacker to already hold a certificate signed by the operator's own CA — this is not a zero-precondition MITM available to an arbitrary internet attacker, but it is a realistic precondition for an insider, a single compromised fleet node, or a compromised issuance pipeline.)

Final severity determination is left to the vendor and depends on the answer to the open question below.

---

5. Open Question for the Vendor

This audit covers the open-source reference implementation (`lightway-core`, `lightway-client`, `wolfssl-rs`) only. **It is not possible, from these repositories alone, to confirm whether the compiled, distributed ExpressVPN application (desktop/mobile) passes `--server-dn` or sets the equivalent internally** when establishing its own VPN connections.

- If it does: the shipped product is not exposed, despite this gap existing in the public reference implementation.
- If it does not: the shipped product inherits this exact gap.

We are asking this directly rather than assuming either way, and would appreciate confirmation either way for an accurate severity rating.

---

6. Suggested Remediation

1. Make hostname/identity validation **mandatory by default** whenever a CA is configured in `lightway-client`, inverting the current opt-in default.
2. At minimum, have `Config::validate()` emit a loud warning (or refuse to start, depending on desired strictness) when `ca_cert` is set but `server_dn` is not.

3. Update the doc comments on `with_server_domain_name_validation` (`lightway-core`) and `checked_domain_name` (`wolfssl-rs`) to state explicitly that omitting the check means the peer's identity is never verified — CA-chain validation alone does not imply "this is the right server," and the current doc comments do not make this risk explicit.

---

7. Artifacts Provided

- `ssl_rs_poc.patch` — the three test functions as a unified diff against `wolfssl-rs/wolfssl/src/ssl.rs`
- `rogue_cert.pem` / `rogue_key.pem` — the self-signed, CA-unrelated certificate used in Test 1
- Full `cargo test -- --nocapture` output for all three tests, reproduced above